

# Journée 3

## Architecture Micro-services

### S12.3 · Intégration JWT dans le service Commande

Nom / Prénom : DANILO Elouan

Équipe métier : **Commande**

## 1. Contexte

Mon service **Commande** (Symfony 7.4 + API Platform) gère les commandes clients, sur les routes `/api/orders` et `/api/customers/{id}/orders`. Pendant le TP S12, c'est l'équipe **Platform** qui s'occupe du service Auth commun : il vérifie les identifiants, émet des JWT signés en **RS256** et publie sa clé publique sur `GET /api/auth/jwks` (le format **JWKS**). De mon côté, je devais faire en sorte que mon service vérifie lui-même ces jetons avant de laisser passer une requête, sans avoir à redemander quoi que ce soit à l'Auth une fois le login effectué.

L'idée est celle du pattern classique aujourd'hui : l'Auth signe avec sa clé privée, chaque service vérifie avec la clé publique. Comme la signature est asymétrique, je n'ai aucun appel synchrone à faire vers l'Auth pour valider un token, ce qui évite d'en faire un goulot d'étranglement.

## 2. Ce que j'ai mis en place

### Récupération de la clé publique via JWKS

Un service `AuthPublicKeyProvider` récupère la JWKS de l'Auth (`JWT_JWKS_URL`) et **met les clés en cache 1 heure**; aucune clé n'est stockée dans un fichier à gérer. Chaque clé JWK (n, e) est convertie en PEM pour permettre la vérification par OpenSSL.

```
$jwks = $httpClient->request('GET', $this->jwksUrl, ['timeout' => 3])->toArray();
foreach ($jwks['keys'] as $jwk) {
    $pems[$jwk['kid']] = JwkConverter::toPem($jwk); // JWK (n,e) -> PEM
}
// mis en cache 1h via CacheInterface
```

En Docker, l'Auth est appelé par son **nom de service** (`http://auth-service:8000`) et non `localhost` (piège S12.4 n°1).

### Vérification de la signature RS256 à chaque requête

Un `JwtAuthenticator` (firewall Symfony) intercepte chaque requête protégée, extrait le `Authorization: Bearer <jwt>`, et vérifie la signature avec la clé publique :

```
if ($header['alg'] !== 'RS256') { throw ...; } // on REFUSE none / HS256
$ok = openssl_verify("$headerB64.$payloadB64",
    Base64Url::decode($signatureB64), $pem, OPENSSL_ALGO_SHA256);
if ($ok !== 1) { throw new RuntimeException('Signature JWT invalide.');
```

### Vérification de l'expiration

Après contrôle de la signature, le claim `exp` est systématiquement vérifié; un token expiré est rejeté en **401** (tolérance d'horloge de 30 s).

```
if (isset($claims['exp']) && time() > ((int) $claims['exp'] + 30)) {
    throw new RuntimeException('JWT expiré.');
```

---

## Endpoints protégés (security.yaml)

Le firewall protège /api/orders et /api/customers; /health et la documentation /api/docs restent publics.

```
# config/packages/security.yaml
firewalls:
  api:
    pattern: ^/api/(orders|customers)
    stateless: true
    custom_authenticators: [App\Security\JwtAuthenticator]
access_control:
  - { path: ^/api/(orders|customers), roles: IS_AUTHENTICATED_FULLY }
```

## 3. Problèmes rencontrés et solutions

---

| Problème                                                                                                                                               | Solution mise en place                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| La librairie <code>firebase/php-jwt</code> était bloquée par un avis de sécurité Composer.                                                             | Vérification RS256 réalisée en <b>PHP/OpenSSL pur</b> ( <code>openssl_verify</code> ), sans dépendance externe ni faille connue.             |
| Le bundle <code>LexikJWT</code> exige la clé publique dans un <b>fichier PEM</b> , incompatible avec « JWKS récupérée automatiquement, sans fichier ». | Bascule sur une <b>vérification JWKS native</b> via un authenticator Symfony maison ; Lexik a été entièrement retiré (Composer + config).    |
| La clé JWKS est un <b>JWK</b> (n/e) alors qu'OpenSSL attend un <b>PEM</b> (piège S12.4 n°2 : « PEM vs JWK incompatibles »).                            | Conversion <code>jwkToPem</code> (encodage ASN.1 DER) écrite à la main et <b>vérifiée identique</b> à la clé d'origine par un test unitaire. |
| En tests, le <code>WebTestCase</code> rebootait le kernel entre deux requêtes, perdant les doubles.                                                    | <code>\$client-&gt;disableReboot()</code> pour conserver les services simulés sur toute la requête.                                          |
| L'opération <code>PATCH</code> renvoyait 415 pour un <code>Content-Type: application/json</code> .                                                     | Ajout de <code>patch_formats</code> acceptant <code>application/json</code> en plus de <code>merge-patch+json</code> .                       |
| <code>openssl_pkey_export</code> échouait sous Windows (config OpenSSL introuvable) pour générer la paire de test.                                     | Génération de la paire RSA de test via le <b>CLI openssl</b> , embarquée dans les fixtures de test.                                          |

---

## 4. Tests bout-en-bout

---

### Suite automatisée

`php vendor/bin/phpunit` → **24 tests, 53 assertions** au vert, dont les 4 cas d'authentification : sans token (401), token malformé (401), token expiré (401), token valide (200).

### Parcours réel contre le service Auth de Platform

Vérifié en direct contre l'Auth réel (`http://10.72.200.53`) et le Catalogue Django.

---

| Requête                                                      | Résultat                                              |
|--------------------------------------------------------------|-------------------------------------------------------|
| GET /api/orders sans token                                   | 401                                                   |
| POST /api/auth/login (Auth) puis GET /api/orders avec Bearer | 200                                                   |
| GET /api/orders avec un token bidon                          | 401                                                   |
| POST /api/orders avec token valide                           | 201, status=PAID, total au prix officiel du Catalogue |
| GET /health (public)                                         | 200                                                   |

La collection Postman (`commande/postman/Commande.postman_collection.json`) automatise ces cas : une requête « Auth - Login » récupère le JWT dans une variable `{{token}}`, réutilisée par toutes les requêtes commande.

## 5. Conclusion

Au final, mon service vérifie tout seul les JWT RS256 émis par l'Auth : il récupère la clé publique via JWKS, contrôle la signature et l'expiration à chaque requête, et ferme les endpoints sensibles à toute requête sans jeton valide. Une fois le login fait, plus aucun appel vers l'Auth n'est nécessaire. Les deux points qui m'ont vraiment posé problème, l'incompatibilité JWK/PEM et la contrainte « sans fichier », ont fini par être réglés avec une vérification JWKS écrite à la main et une conversion JWK vers PEM maison, le tout couvert par les tests et un parcours bout-en-bout réussi contre l'infrastructure réelle.

# A. Annexes : captures d'écran

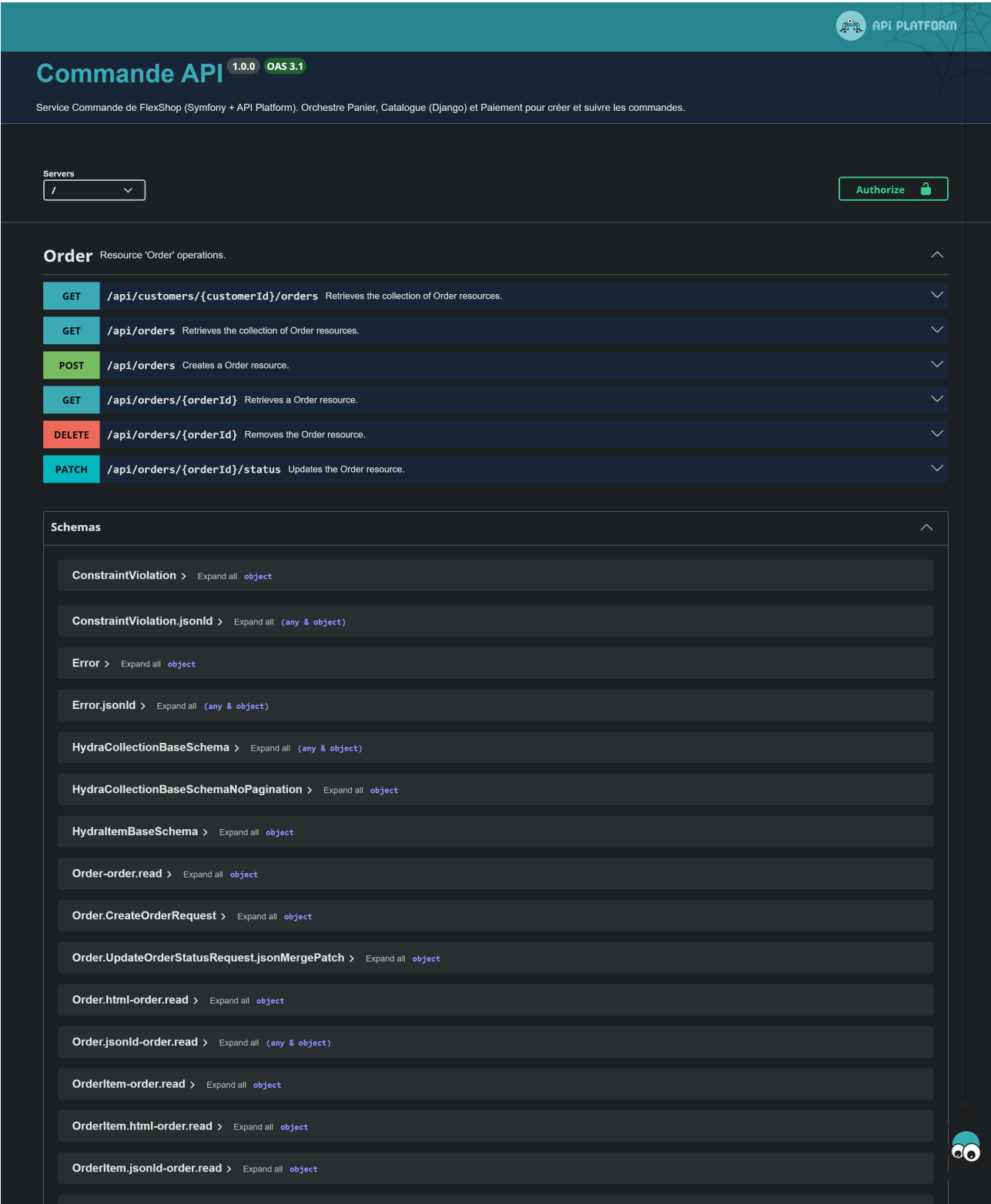


FIGURE 1 – Annexe A. Swagger UI du service Commande (/api/docs) : les six opérations du contrat et le bouton Authorize (JWT).

```
Parcours JWT bout-en-bout - Commande / Auth 10.72.200.53

# Parcours JWT bout-en-bout - Commande vs Auth reel 10.72.200.53
# (2026-07-01 12:27)

$ curl -i http://localhost:8002/api/orders # SANS token
HTTP/1.1 401 Unauthorized
Content-Type: application/json
{"code":401,"message":"En-t\u00eate Authorization Bearer manquant."}

$ curl -X POST http://10.72.200.53/api/auth/login -d '{"username":"alice","password":"customer"}'
{"token":"eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ... (RS256, 861c) ...wDVTkeEXxKJie0gg"}
# payload decode : {"iat": 1782901647, "exp": 1782905247, "sub": "1", "roles": ["ROLE_USER"], "username": "alice"}
# exp : 2026-07-01 11:27 UTC (alg=RS256, sub=1, roles=ROLE_USER)

$ curl -H 'Authorization: Bearer $TOKEN' http://localhost:8002/api/orders # token VALIDE
HTTP/1.1 200 OK
totalItems = 15 → 200 OK, cle publique JWKS verifiee

$ curl -H 'Authorization: Bearer TOKEN_BIDON' http://localhost:8002/api/orders # invalide
HTTP/1.1 401 Unauthorized
{"code":401,"message":"JWT invalide : Signature JWT invalide."}
```

**FIGURE 2 – Annexe B. Parcours JWT bout-en-bout en direct contre l'Auth réel (10.72.200.53) :** GET /api/orders sans token → 401; POST /api/auth/login → JWT RS256 (payload sub, roles, exp décodés); même requête avec Bearer valide → 200 (clé publique JWKS vérifiée); token bidon → 401 (« Signature JWT invalide »).

```
php vendor/bin/phpunit --testdox (service Commande)

PHPUnit 11.5.55 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.3.31
Configuration: C:\Users\oasio\Desktop\S6\microservice\J1\commande\phpunit.xml.dist

..... 24 / 24 (100%)

Time: 00:01.398, Memory: 44.00 MB

Jwk Converter (App\Tests\Security\JwkConverter)
✓ Jwk to pem reproduces original public key

Jwt Auth (App\Tests\Api\JwtAuth)
✓ Protected endpoint without token returns 401
✓ Protected endpoint with bogus token returns 401
✓ Protected endpoint with expired token returns 401
✓ Protected endpoint with valid token returns 200
✓ Health endpoint is public

Order Api (App\Tests\Api\OrderApi)
✓ List orders responds ok
✓ Create order returns 201 and is paid
✓ Create order with unknown cart returns 422
✓ Create order with missing customer returns 422
✓ Get order by reference returns 200
✓ Get unknown order returns 404
✓ Patch status valid transition returns 200
✓ Patch status unknown value returns 400
✓ Patch status forbidden transition returns 409
✓ Cancel order returns 200 and cancelled
✓ Cannot cancel shipped order
✓ List customer orders
✓ Idempotency key returns same order
✓ Payment refused keeps order failed

Order Status Transition (App\Tests\Entity\OrderStatusTransition)
✓ Nominal flow is allowed
✓ Backward transition is forbidden
✓ Cancellation rules
✓ Cancel transition allowed while not shipped

OK (24 tests, 53 assertions)
```

**FIGURE 3** – Annexe C. Suite PHPUnit : 24 tests au vert (dont les 4 cas JWT : sans token 401, token bidon 401, token expiré 401, token valide 200) et la conversion JWK → PEM.